Reviewer Pack — Hodge Structure Rank of Algebraic Curves vs BP ReadTwice Cir…

Entry 086354827787 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 18:16:30 UTC

Hodge Structure Rank of Algebraic Curves vs BP ReadTwice Circuit Threshold

Entry ID: `086354827787` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 18:16:30 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: Branching Program Complexity

Statement:

> ['For every n-vertex graph G, the Hodge structure rank of its associated algebraic curve C(G) is polynomially related to the BP ReadTwice circuit threshold for G.', 'Specifically, if k = BP_ReadTwice(G), then the Hodge structure rank of C(G) is bounded above by a polynomial function of k and below by O(k).', 'This relationship holds for all graphs G with n ≤ 40.']

Rationale (proposer's reasoning):

> ["Hodge theory provides a powerful tool to study algebraic varieties, including curves. The Hodge structure rank captures the complexity of the variety's geometry.", 'Branching programs are a model of computation that can simulate deterministic circuits with a polynomial overhead. Understanding the relationship between the geometric complexity of graphs and the computational complexity captured by BP ReadTwice circuits could shed light on the limits of efficient computation.', 'The conjecture suggests a bridge between these two fields, potentially revealing new insights into computational complexity.']

Taxonomy category: `COMPUTATIONAL_ALGEBRAIC_GEOMETRY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `4607bead5f905f97`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the estimated Hodge structure rank of C(G) is within a polynomially bounded range of BP_ReadTwice circuit threshold k, specifically |rank(C(G)) - k| ≤ 3 * poly(k), for all graphs G with n ≤ 40.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge structure rank" AND "algebraic curve" AND BP ReadTwice - "complexity theory" AND "branching program" AND Hodge structure rank - "polynomial relationship" AND algebraic geometry AND BP_ReadTwice circuit threshold

Top relevant hits considered: - [http://arxiv.org/abs/2211.07055v3] Geometric complexity theory for product-plus-power - [http://arxiv.org/abs/2604.00746v2] An Unconditional Barrier for Proving Multilinear Algebraic Branching Program Lower Bounds - [http://arxiv.org/abs/2003.04834v1] Algebraic Branching Programs, Border Complexity, and Tangent Spaces - [http://arxiv.org/abs/2412.20630v2] Computing with D-Algebraic Sequences - [http://arxiv.org/abs/2304.09675v2] Operations for D-Algebraic Functions - [http://arxiv.org/abs/2008.00227v1] Traces, symmetric functions, and a raising operator

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

# Helper functions for linear algebra
def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate non-pivot elements
        pivot = matrix[i][i]
        for k in range(n):
            matrix[i][k] /= pivot

        for j in range(n):
            if i != j:
```

```
                factor = matrix[j][i]
                for k in range(n):
                    if k < i:  # Avoid modifying the pivot row
                        matrix[j][k] -= factor * matrix[i][k]

def rank(matrix):
    n, m = len(matrix), len(matrix[0])
    gaussian_elimination(matrix)
    rank = 0
    for i in range(n):
        if any(matrix[i][j] != 0 for j in range(m)):
            rank += 1
    return rank

# Function to generate a random graph as an adjacency matrix
def generate_graph(n):
    graph = [[0]*n for _ in range(n)]
    edges = set()
    while len(edges) < n*(n-1)//2:
        u, v = random.sample(range(n), 2)
        if u != v and (u, v) not in edges and (v, u) not in edges:
            graph[u][v] = 1
            graph[v][u] = 1
            edges.add((u, v))
    return graph

# Function to compute the BP ReadTwice circuit threshold for a graph
def bp_read_twice(graph):
    n = len(graph)
    max_k = 0
    for k in range(1, n+1):
        if all(sum(graph[u][v] for u in range(k) if v >= k) == sum(graph[u][v] for u in range(n-k)
            max_k = k
    return max_k

# Function to run a single trial with a given seed
def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random graph
    n = random.randint(5, 40)
    graph = generate_graph(n)

    # Compute the BP ReadTwice circuit threshold
    k = bp_read_twice(graph)

    # Estimate the Hodge structure rank (simplified for testing purposes)
    matrix = [[graph[i][j] for j in range(n)] for i in range(n)]
    rank_value = rank(matrix)

    # Check if the conjecture holds
    conjecture_holds = abs(rank_value - k) <= 3 * k**2

    return {
        "metric_name": "Rank vs DPLL Heig",
        "metric_value": rank_value,
```

```
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"Graph with n={n}, k={k}, rank={rank_value}
    }

# Main function to run multiple trials
if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    # Compute mean and standard deviation of metric_value
    total_metric_value = sum(result["metric_value"] for result in results)
    mean_metric_value = total_metric_value / len(results)
    variance = sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(
    std_metric_value = math.sqrt(variance)

    # Compute fraction of seeds where conjecture holds
    support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11504ca4.py", line 104, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11504ca4.py", line 84, in run_trial
    rank_value = rank(matrix)
                 ^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11504ca4.py", line 43, in rank
    gaussian_elimination(matrix)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11504ca4.py", line 32, in gaussian_elim
    matrix[i][k] /= pivot
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means the support condition could not be verified. | next: Re-run the test without crashing to verify if the estimated Hodge structure rank of C(G) is within the polynomially bounded range of BP_ReadTwice circuit threshold k.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13165
2	propose	ollama_remote	glm4:latest	0	0	11939
3	propose	ollama_remote	glm4:latest	0	0	10326
4	preregistration	ollama_remote	glm4:latest	0	0	6088
5	novelty	ollama_remote	glm4:latest	0	0	4734
6	novelty	ollama_remote	glm4:latest	0	0	6983
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39975
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10451
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10438
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12552
11	judge	ollama_remote	glm4:latest	0	0	11297

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 137948 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/086354827787.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/086354827787.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/086354827787.tar.gz` (if generated)